Title: Front-End & Back-End Connectivity (CRUD Operations)

Objective: To be able to create database connectivity and integration between a front-end application and a relational database.

.

Total Marks: 10

Expected Outcome of Experiment:

CO3: Utilize SQL for Relational Database Operations

Resources used:

Postgresql for backend, **Frontend-** Any platform

Student Details

Student 1 Name: Ashwera Hasan

Student 1 Roll No: 16010124107

Student 2 Name: Navjyot Hari

Student 2 Roll No: 16010124106

Class / Division: B2

Subject: RDBMS Lab

Faculty Name: Prof Megha Sharma

Date of Submission: 15.04.2026

1. Problem Statement

Write the problem statement assigned to your group.

Design and implement a database system to efficiently store, manage, and retrieve structured data in a banking system. The system should support operations like insertion, deletion, updating, and querying while ensuring data integrity and minimizing redundancy. It must demonstrate concepts such as normalization, indexing, and relationships between tables to improve performance and consistency.

2. Technologies Used

Front-End Technology: HTML, CSS

Back-End / Server Language: PHP

Database Used: MySQL

Tool/IDE Used: VS Code

3. Database Design

Write the SQL queries used for creating tables and inserting initial records.

CREATE TABLE Queries:

```
CREATE TABLE Customers (
customer_id INT PRIMARY KEY AUTO_INCREMENT,
name VARCHAR(100),
email VARCHAR(100),
phone VARCHAR(15)
);

CREATE TABLE Accounts (
account_id INT PRIMARY KEY AUTO_INCREMENT,
customer_id INT,
account_type VARCHAR(20),
balance DECIMAL(10,2),
FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);
```

```
CREATE TABLE Transactions (
transaction_id INT PRIMARY KEY AUTO_INCREMENT,
account_id INT,
amount DECIMAL(10,2),
transaction_type VARCHAR(20),
transaction_date DATE,
FOREIGN KEY (account_id) REFERENCES Accounts(account_id)
);
```

INSERT Queries:

```
INSERT INTO Customers (name, email, phone) VALUES
('Amit Sharma', 'amit@gmail.com
', '9876543210'),
('Neha Verma', 'neha@gmail.com
', '9123456780');

INSERT INTO Accounts (customer_id, account_type, balance) VALUES
(1, 'Savings', 5000.00),
(2, 'Current', 12000.00);

INSERT INTO Transactions (account_id, amount, transaction_type, transaction_date)
VALUES
(1, 1000.00, 'Deposit', '2026-04-10'),
(1, 500.00, 'Withdrawal', '2026-04-11'),
(2, 2000.00, 'Deposit', '2026-04-12');
```

---

4. Database Connectivity

Explain how the front-end is connected with the database. Mention the connection code or logic used (for example: PHP with MySQL, Python with PostgreSQL, etc.).

The front-end (HTML/CSS) sends user data through forms to the back-end using HTTP requests. The back-end (PHP) connects to the MySQL database using a database connector and executes SQL queries.

Example connection logic in PHP:

```
$conn = mysqli_connect("localhost", "root", "", "bank_db");


if (!$conn) {

   die("Connection failed: " . mysqli_connect_error());

}
```

The PHP script receives input, runs queries (INSERT, SELECT, etc.), and returns results to the front-end for display.

5. CRUD Operations Implementation

Create Operation:

New records are inserted through a form in the front-end where the user enters details. The data is sent to the PHP back-end using the POST method. PHP processes the input and executes an INSERT SQL query to store the data in the MySQL database.

Read Operation:

Records are retrieved using a SELECT SQL query in PHP. The fetched data is processed and displayed on the front-end using HTML tables, allowing users to view stored information.

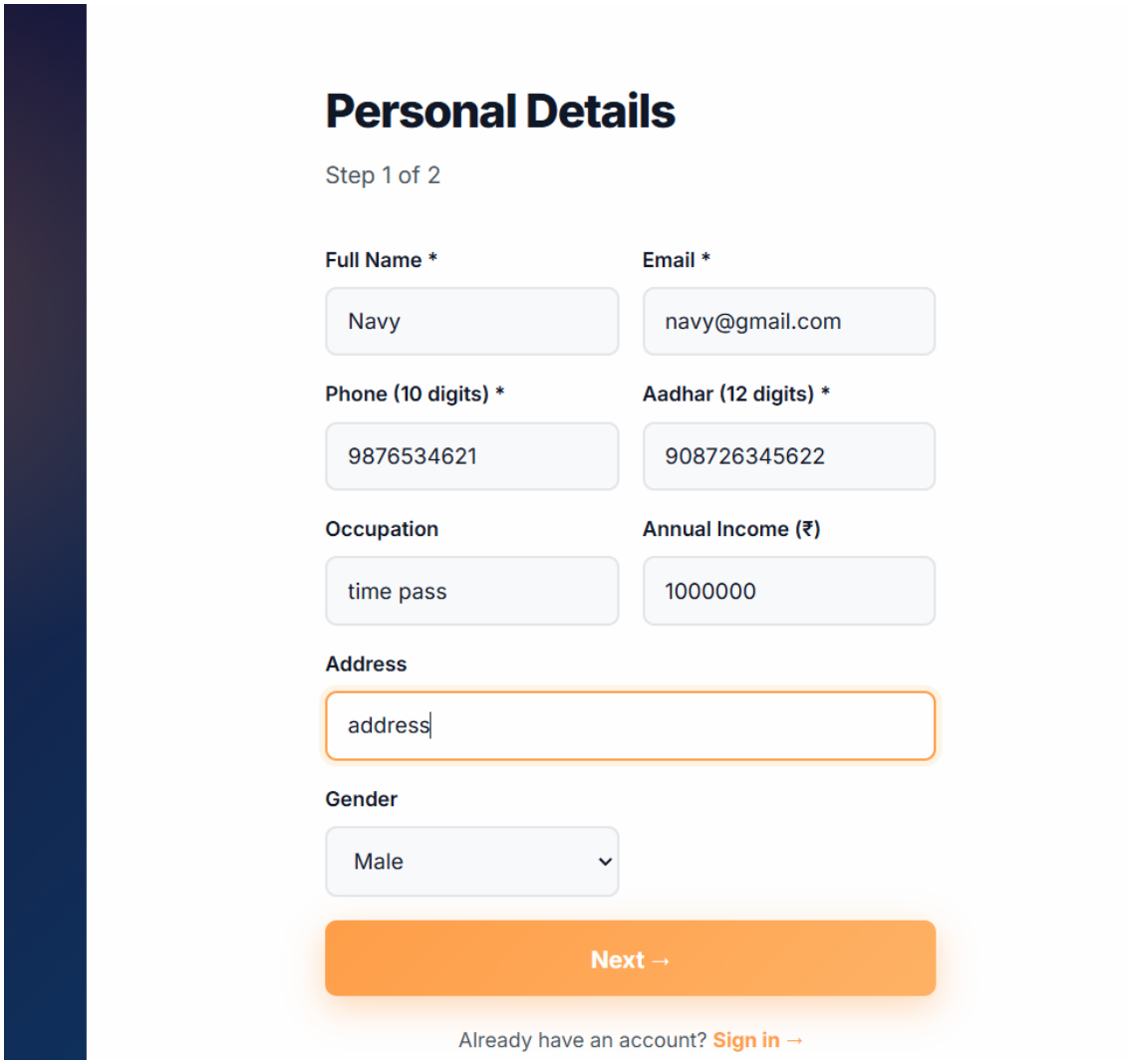
Update Operation:

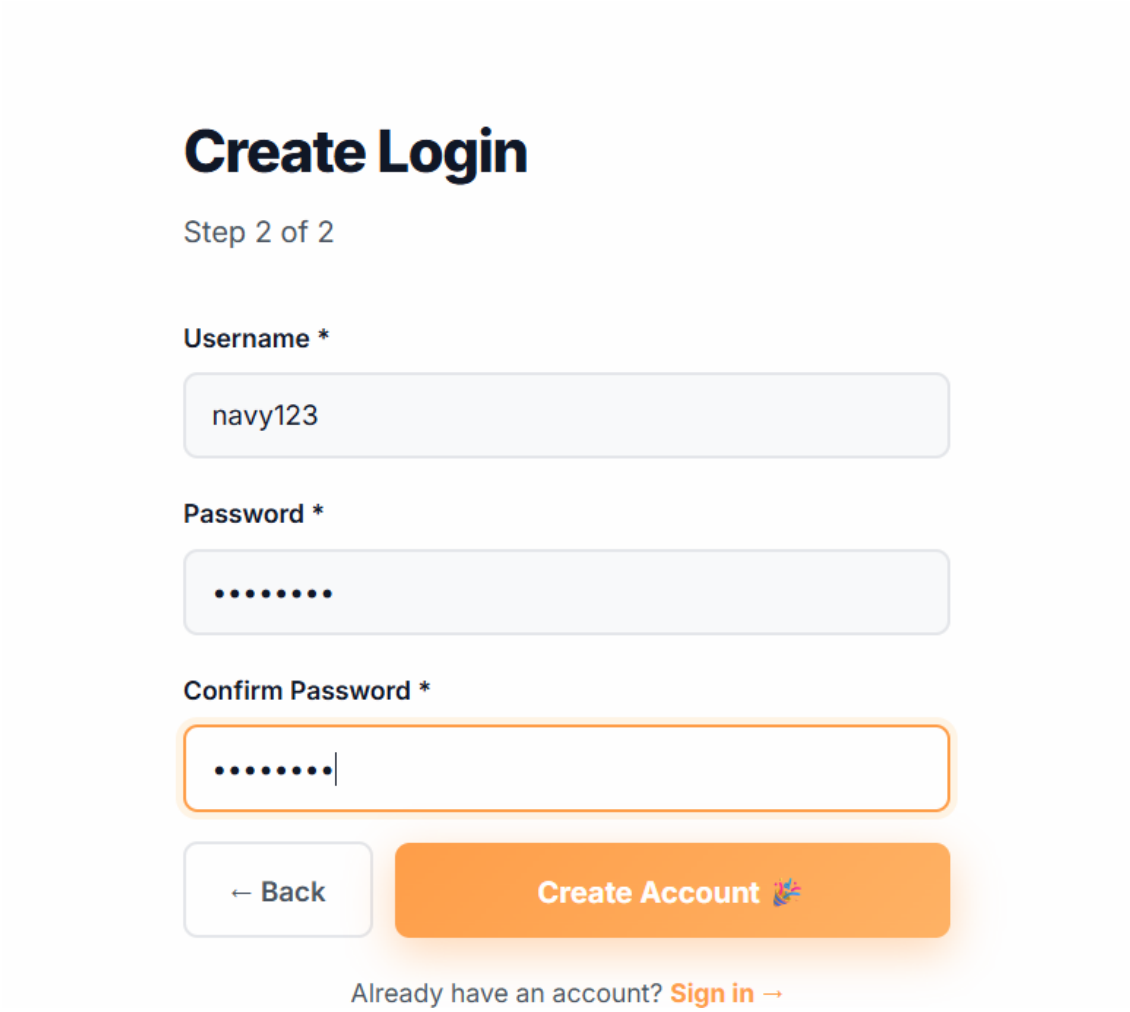
Existing records are modified through an update form. The user edits the required fields, and the updated data is sent to PHP. An UPDATE SQL query is executed to modify the corresponding record in the database.

Delete Operation:

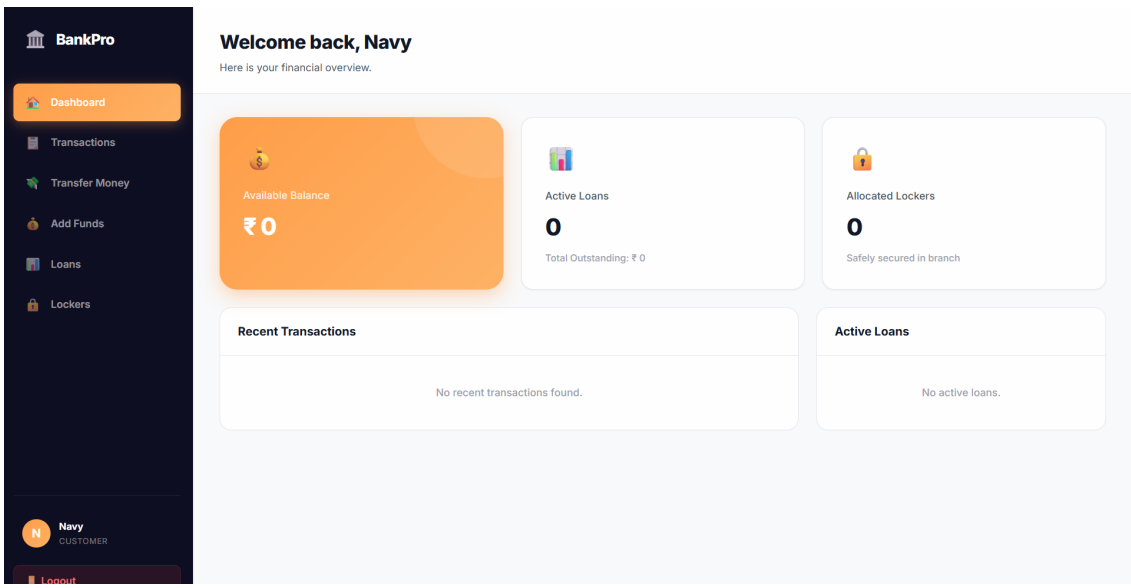
Records are deleted when the user selects a delete option. The request is sent to PHP, which executes a DELETE SQL query to remove the selected record from the database permanently.

6. Screenshots



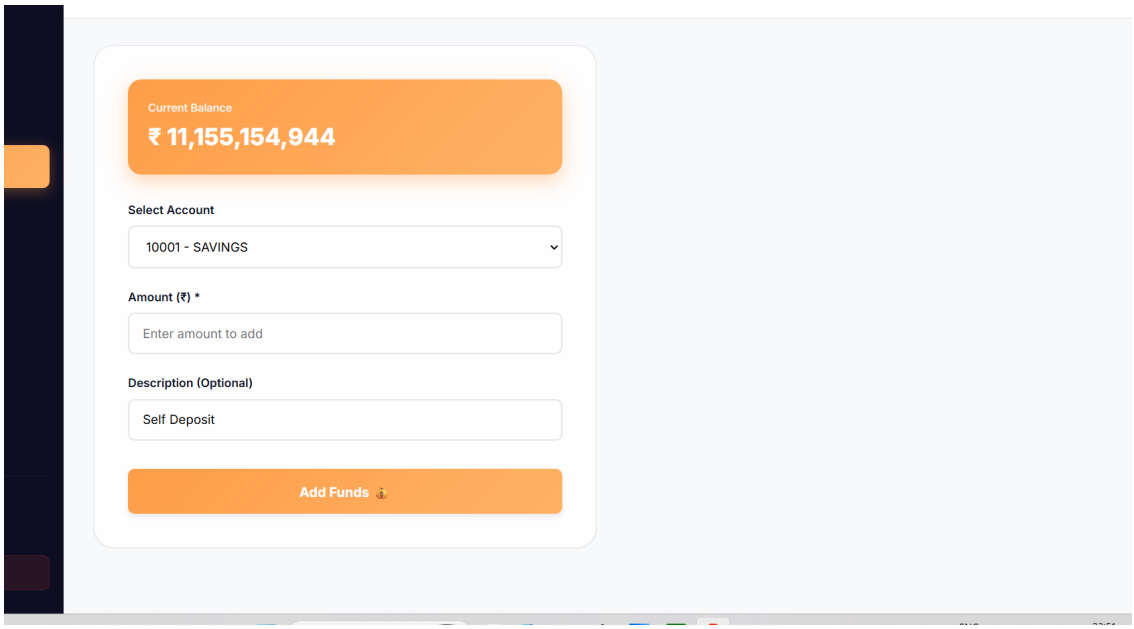


• Display/View (Read) operation

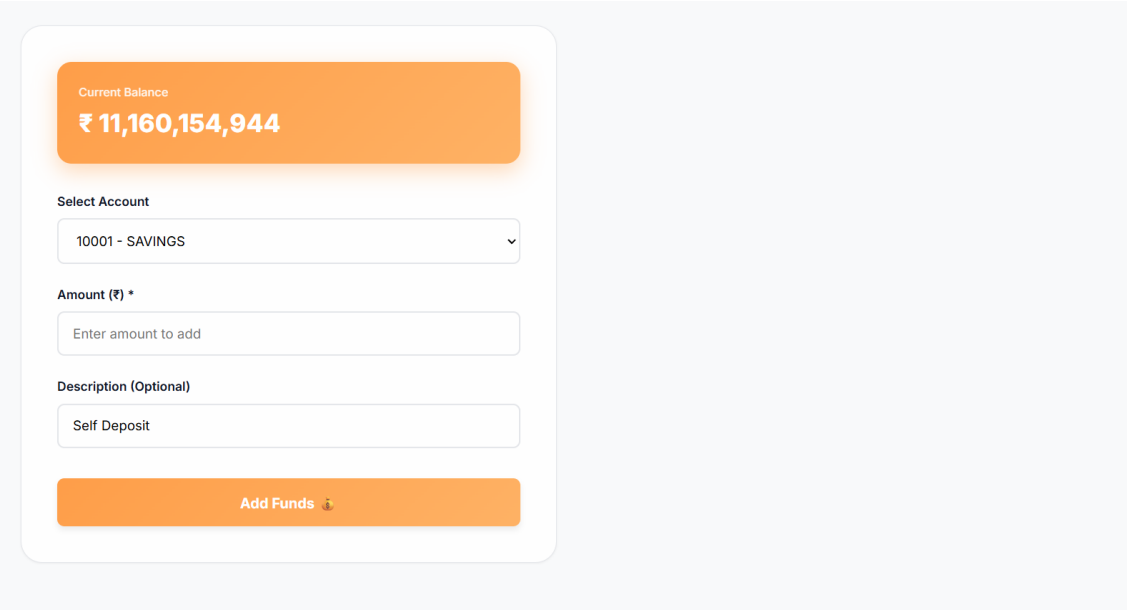


• Update operation

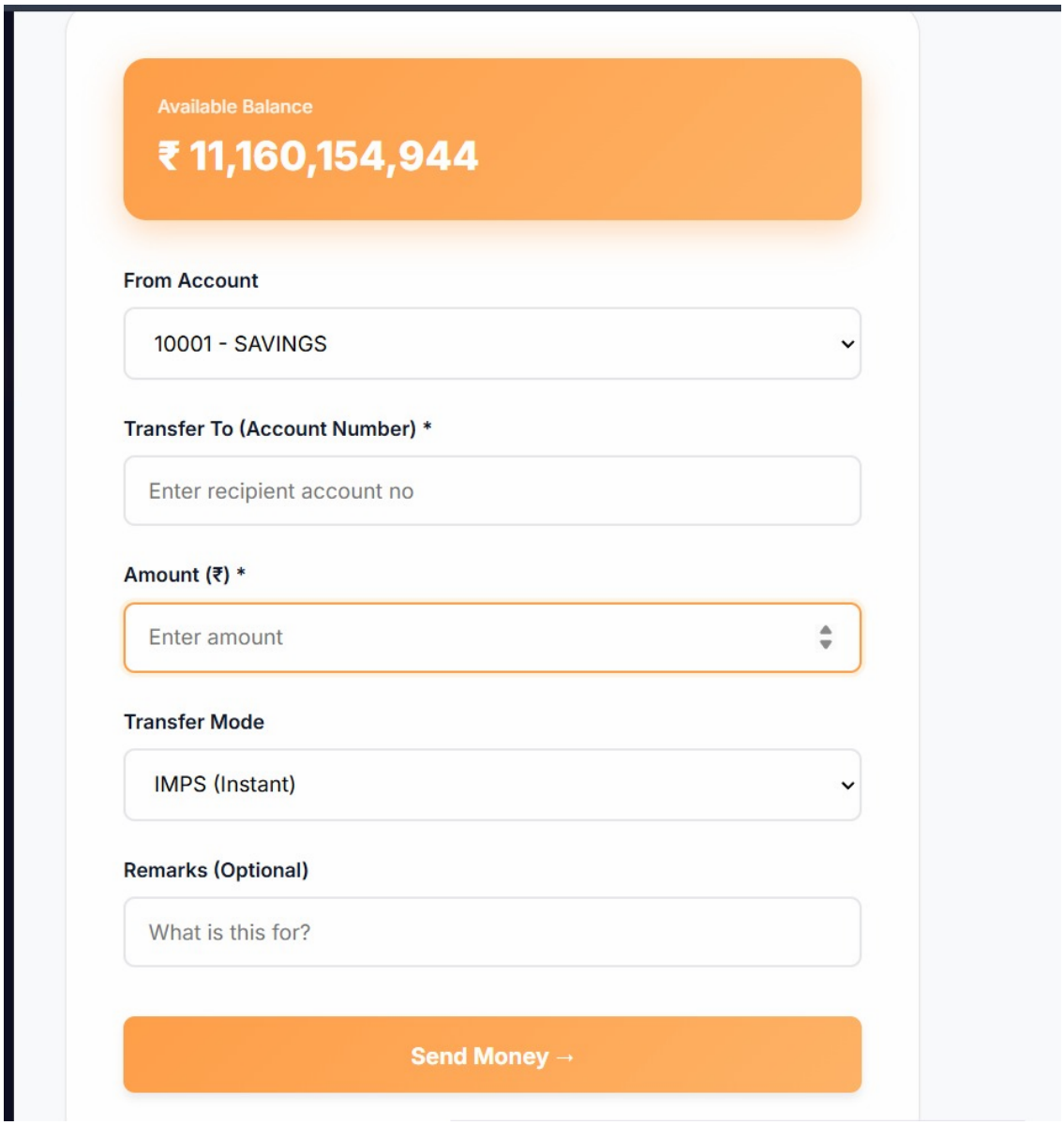
before:

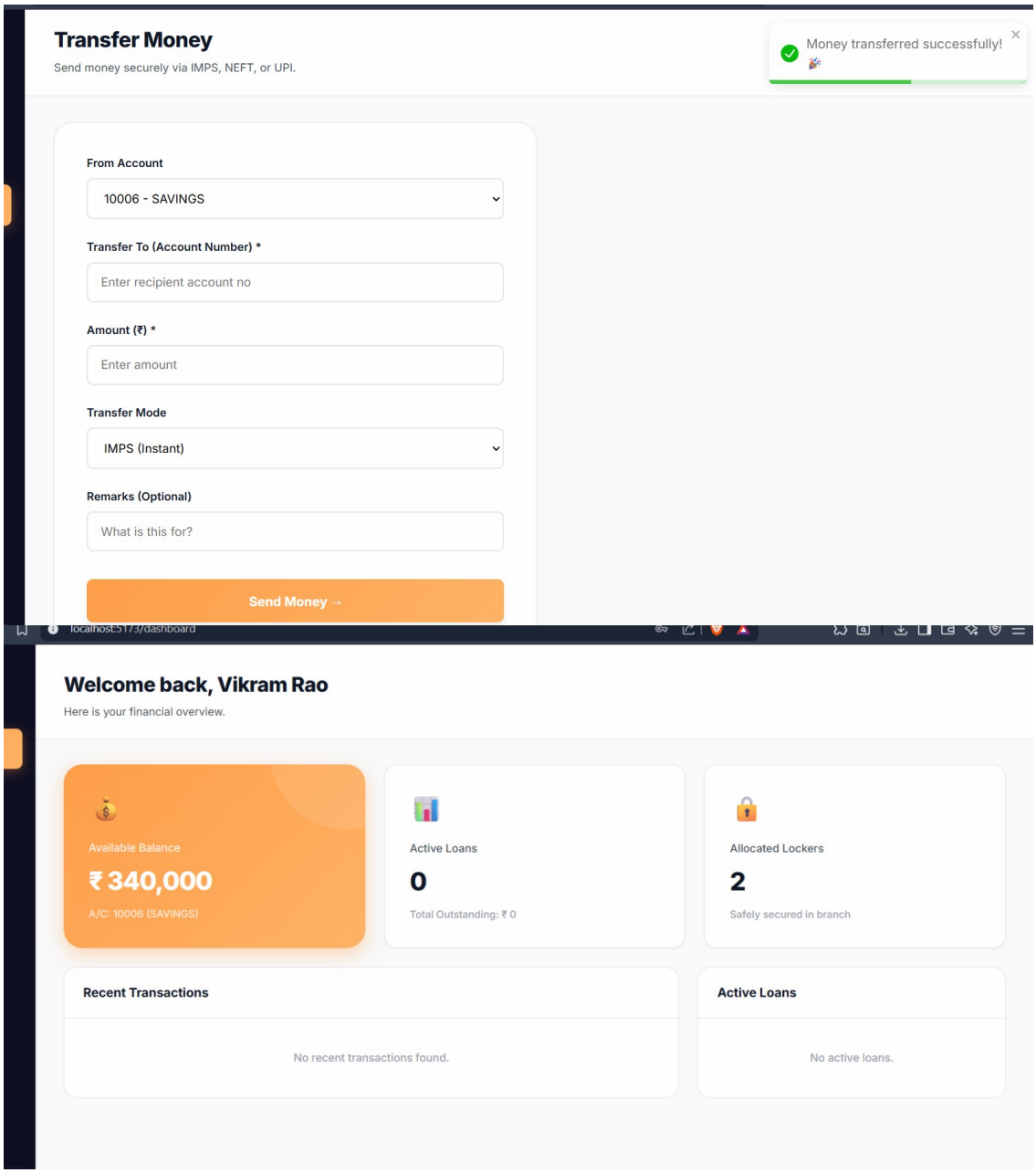


after:



• Delete operation


Available Balance
₹ 11,160,154,944
From Account
10001 - SAVINGS
Transfer To (Account Number) *
Enter recipient account no
Amount (₹) *
Enter amount
Transfer Mode
IMPS (Instant)
Remarks (Optional)
What is this for?
Send Money →



7. Conclusion

This experiment helped in understanding how front-end and back-end are connected to a database. It showed how user inputs are processed and stored using SQL queries. CRUD operations helped in learning how to insert, retrieve, update, and delete data efficiently. Overall, it improved understanding of database connectivity, data handling, and building dynamic web applications.